

CHAPTER 1

INTRODUCTION

Medication adherence plays a critical role in effective healthcare management, especially for elderly individuals, chronically ill patients, and people with busy lifestyles who often struggle to take medicines at the correct time. Missing doses, taking incorrect quantities, or forgetting medication schedules can lead to severe health complications and reduced treatment effectiveness. With the advancement of IoT technologies, there is a growing demand for automated systems that can assist patients in maintaining proper medication routines without constant manual supervision. In this context, smart medicine dispensing systems offer an innovative and reliable solution to ensure timely and accurate delivery of prescribed medicines. This project, Smart Medicine Dispenser Using ESP8266, is developed to address the challenges associated with manual medication management. It provides an automated, user-friendly, and Wi-Fi-enabled system capable of reminding the user and dispensing medicines at predefined times. At the core of this system is the ESP8266 NodeMCU, which serves as the central controller and enables wireless connectivity. By connecting to Wi-Fi, the device can synchronize the current time using NTP (Network Time Protocol) servers, ensuring accurate timing for medication reminders. The user can configure up to three daily medicine schedules through a web interface hosted by the ESP8266, making the system convenient and easy to set up. To ensure precise timekeeping and backup storage, the system integrates an RTC (Real-Time Clock) module such as DS3231 or DS1307. Even during power interruptions, the RTC retains the programmed schedule, thereby maintaining complete reliability. A 16×2 LCD display (I2C) provides real-time information, including the current time and upcoming dosage reminders, enhancing user interaction and awareness. For physical dispensing, a servo motor is used to rotate and release the correct medicine from a designated slot or container. Whenever it is time for the user to take medicine, the servo activates, and an audible alert is generated by a buzzer.

To ensure that the user acknowledges the dose, a push button is included as a confirmation mechanism. The buzzer continues to sound until the user presses the button, thereby preventing missed doses. Additional components such as jumper wires, a 5V power supply, and organized medicine slots complete the system's hardware requirements. Overall, the Smart Medicine Dispenser provides an efficient, automated, and dependable solution for medication management.

1.1 PROBLEM STATEMENT

Many patients, especially the elderly and chronically ill, frequently forget to take their medicines on time or take incorrect doses, which leads to ineffective treatment and serious health risks. Traditional pillboxes only store medicines and do not provide reminders, monitoring, or confirmation of intake. Caregivers and family members also have no reliable way to track whether the patient followed the medication schedule. This lack of real-time monitoring increases the chances of missed doses, overdose, and medical emergencies. Therefore, there is a need for an intelligent and automated system that can remind patients, ensure correct dosage, and update caregivers instantly. An IoT-based smart medicine box can fulfill this need by offering timely alerts, automatic tracking, and cloud-based monitoring to improve medication adherence.

1.2 OBJECTIVES

The main objectives of the project is as follows:

1. Automated Medication Reminders

To design a system that provides timely audio, visual, or mobile notifications to remind users to take their medications.

2. Dose Tracking and Monitoring

To monitor whether the user has taken the correct medication at the scheduled time using sensors (e.g., compartment open detection).

3. IoT Connectivity

To enable real-time communication between the medicine box and a mobile/web application using IoT technologies like Wi-Fi or Bluetooth.

4. User and Caregiver Alerts

To send alerts to caregivers or family members in case of missed doses or irregular medication behaviour.

5. User-Friendly Interface

To provide a simple and intuitive interface (on-device and via app) for users of all age groups, especially the elderly.

1.3 SCOPE OF THE STUDY

The scope of this study focuses on the design and development of an intelligent, IoT-based medicine dispensing system that ensures timely and accurate medication intake for users. The project covers the creation of a functional prototype using an ESP8266 microcontroller integrated with a real-time clock (RTC), buzzer, LCD display, and automated dispensing mechanism. The system is designed to store medicine schedules, alert the user through sound and visual notifications, and dispense the correct dosage at the programmed time. This study also includes the development of a user interface for setting timings and monitoring the status of medicine intake. The research is limited to solid-form medications such as tablets or capsules and is intended primarily for home use, especially for elderly individuals, chronically ill patients, and those requiring strict medication routines.

The system focuses on automation, reliability, and ease of use rather than advanced medical diagnostics. Network-based remote monitoring using Wi-Fi may be included, but complex cloud-based analytics or hospital integration remain outside the scope. The prototype aims to demonstrate how IoT technology can improve medication adherence, reduce human error, and offer a supportive tool for caregivers, without replacing professional medical advice or clinical systems.

CHAPTER 2

LITERATURE SURVEY

PAPER 1

“IoT Smart Medicine Box” Lee et al., —International Journal of Research in Engineering, Science and Management (IJRESM), 2018.

This paper discusses the design and implementation of an IoT-based smart medicine box that uses sensors and RFID technology to ensure proper medication intake. Smart Medication. The core component of this system is an intelligence medicine box equipped with wireless connectivity and integrated with an android application, aiming to improve communication between patients and healthcare providers. The paper emphasizes the use of cloud-based storage for logging medication history and improving remote health monitoring. This concept strongly supports your Intelligent Medicine Box, as it showcases how IoT can be leveraged to create a proactive, connected, and automated medication management system that enhances patient care and safety.

PAPER 2

“Management System”.Patel et al., —International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET), 2019.

This study presents an IoT-based medication management system where a smart medicine box is connected to a cloud server. These systems typically integrate hardware components like microcontrollers, sensors, and wireless communication modules with software applications, often on smartphones, to provide a comprehensive medication management solution. If the medicine is not taken within the scheduled window, the system records this event and sends notifications to caregivers via IoT modules such as GSM or Wi-Fi. The paper also emphasizes features like data logging, which allows tracking of medication history over time. This aligns closely with your Intelligent Medicine Box, supporting the concept of using IoT-based reminders, real-time alerts, and remote monitoring to enhance medication adherence and improve healthcare outcomes.

PAPER 3

“Personalized Medication Systems”. Zhang et al., —Design of a Smart Medical Box for Automatic Pill Dispensing and Health Monitoring, 2021.

This paper presents a smart medicine box that customizes medication reminders based on the patient's medication schedule. It includes a voice reminder system to alert users when its time to take their medicine and Bluetooth enabled mobile application for scheduling and monitoring. This personalized IOT based system enhances health monitoring, reduces human error, and promotes consistent medication management. The paper highlights the importance of AI integration and machine learning for analyzing user behavior and optimizing medication plans. This approach strongly supports your Intelligent Medicine Box, showing how IoT can not only automate reminders and alerts but also enable adaptive, personalized medication management that enhances patient engagement and treatment effectiveness.

PAPER 4

Medication Management using Smart Pill Box and IoT" — International Journal of Engineering Research & Technology (IJERT), 2020

This study introduces an automated pill dispenser designed to assist individuals, particularly the elderly, in managing their medication schedules effectively. The system integrates a microcontroller-based pill box with IoT capabilities to provide timely reminders and monitor medicine intake. Using a Real-Time Clock (RTC), the system triggers audio-visual alerts (like buzzers and LEDs) at scheduled times to remind users to take their medication. It includes compartments for organizing pills according to dosage times (morning, afternoon, night). If the patient does not respond by pressing a button or opening the compartment, the system can send alerts to caregivers or family members via Wi-Fi or GSM module. The data can also be uploaded to the cloud for remote monitoring. Overall, the paper emphasizes the role of IoT in enhancing patient care by automating reminders, reducing human error, and enabling real-time tracking of medication intake.

PAPER 5

"IoT Based Smart Medicine Box for Elderly People" — International Journal of Innovative Research in Computer and Communication Engineering (IJIRCCE), 2021

These systems are designed to enhance medication adherence among elderly patients by providing timely reminders and ensuring the correct dosage is taken. They typically integrate various technologies to create an efficient and user-friendly experience. The medicine box is divided into multiple compartments, each linked to a specific time slot. When it's time to take a dose, the system generates reminders through buzzers or LED indicators, ensuring the user is notified. If the medicine is not taken on time, the system logs this and sends alerts to caregivers or family members via the internet or SMS. The paper highlights the significance of using IoT for real-time tracking, improved medication adherence, and remote health monitoring, making it especially helpful for elderly people who may forget their schedules. This aligns with the goal of your Intelligent Medicine Box, combining automation, IoT, and user-friendly interfaces to promote better health outcomes.

PAPER 6

"Smart Medicine Box for Medication Management using IoT" — International Research Journal of Engineering and Technology (IRJET), 2020

These systems are designed to assist patients, particularly the elderly or those with chronic conditions, in managing their medication schedules effectively. The medicine box contains separate compartments for different dosage times, helping organize multiple medications. The system also includes IoT connectivity, typically using Wi-Fi or GSM, to send real-time alerts and updates to caregivers or family members if a dose is missed. The paper emphasizes the importance of automating medication reminders and enabling remote monitoring, making the solution highly suitable for elderly or forgetful patients. This directly supports the concept behind your Intelligent Medicine Box, which aims to combine automated reminders, dose tracking, and IoT communication to enhance medication compliance.

CHAPTER 3

HARDWARE DESCRIPTION

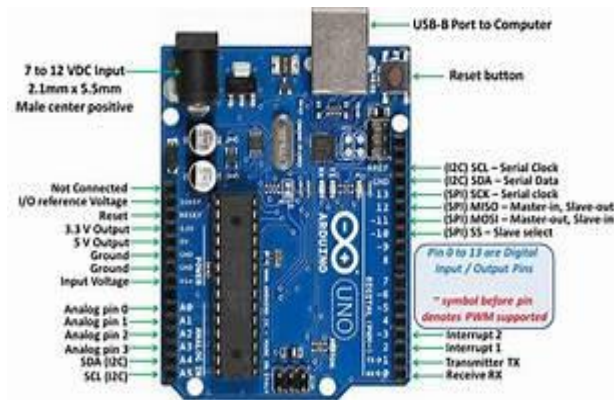
3.1 ARDUINO



Fig 3.1.1- Arduino Uno

Arduino Uno Board Pin Configuration Power Supply the Arduino Uno power supply can be done with the help of a USB cable or an external power supply. The external power supplies mainly include AC to DC adapter otherwise a battery. The adapter can be connected to the Arduino Uno by plugging into the power jack of the Arduino board. Similarly, the battery leads can be connected to the Vin pin and the GND pin of the POWER connector. The suggested voltage range will be 7 volts to 12 volts. **Input & Output** The 14 digital pins on the Arduino Uno can be used as input & output with the help of the functions like pinMode(), digitalWrite(), & Digital Read(). **Pin1 (TX) & Pin0 (RX) (Serial):** This pin is used to transmit & receive TTL serial data, and these are connected to the ATmega8U2 USB to TTL Serial chip equivalent pins. **Pin 2 & Pin 3 (External Interrupts):** External pins can be connected to activate an interrupt over a low value, change in value. **Pins 3, 5, 6, 9, 10, & 11 (PWM):** This pin gives 8-bit PWM o/p by the function of analogWrite(). **SPI Pins (Pin-10 (SS), Pin-11 (MOSI), Pin-12 (MISO), Pin-13 (SCK):** These pins maintain SPI communication, even though offered by the fundamental hardware, is not presently included within the Arduino language. **Pin-13(LED):** The inbuilt LED can be connected to pin-13 (digital pin). As the HIGH-value pin, the light emitting diode is activated, whenever the pin is LOW. **Pin-4 (SDA) & Pin-5 (SCL) (I2C):** It supports TWI-communication with the help of the Wire library. **AREF (Reference Voltage):** The reference

voltage is for the analog i/ps with `analogReference()`. Reset Pin: This pin is used for reset (RST) the microcontroller. Memory The memory of this Atmega328 Arduino microcontroller includes flash memory-32 KB for storing code, SRAM-2 KB EEPROM-1 KB.



3.1.2. Arduino Uno Pin Diagram

Communication

Arduino software contains a wired library for simplifying the utilization of the I2C The Arduino Uno ATmega328 offers UART TTL-serial communication, and it is accessible on digital pins like TX (1) and RX (0). The software of an Arduino has a serial monitor that permits easy data. There are two LEDs on the board like RX & TX which will blink whenever data is being broadcasted through the USB. A SoftwareSerial library permits for serial communication on Arduino Uno digital pins and the ATmega328P supports TWI (I2C) as well as SPI-communication.

3.2 ESP8266 NodeMCU microcontroller

NodeMCU is a low-cost open source IoT platform. It initially included firmware which runs on the ESP8266 Wi-Fi SoC from Espressif Systems, and hardware which was based on the ESP-12 module.[6][7] Later, support for the ESP32 32-bit MCU was added. The name "NodeMCU" combines "node" and "MCU" (micro-controller unit).[8] Strictly speaking, the term "NodeMCU" refers to the firmware rather than the associated development kits. Both the firmware and prototyping board designs are open source.[8] The firmware uses the Lua scripting language. The firmware is based on the eLua project, and built on the Espressif Non-

OS SDK for ESP8266. It uses many open source projects, such as lua cJSON[9] and SPIFFS.[10] Due to resource constraints, users need to select the modules relevant for their project and build a firmware tailored to their needs. Support for the 32-bit ESP32 has also been implemented. The prototyping hardware typically used is a circuit board functioning as a dual in-line package (DIP) which integrates a USB controller with a smaller surface-mounted board containing the MCU and antenna. The choice of the DIP format allows for easy prototyping on breadboards. The design was initially based on the ESP-12 module of the ESP8266, which is a Wi-Fi SoC integrated with a Tensilica Xtensa LX106 core, widely used in IoT applications.



Fig 3.2.1- NODE MCU ESP8266 development board

The good thing about ESP32, like ESP8266 is its integrated RF components like Power Amplifier, Low-Noise Receive Amplifier, Antenna Switch, Filters and RF Balun. This makes designing hardware around ESP32 very easy as you require very few external components.

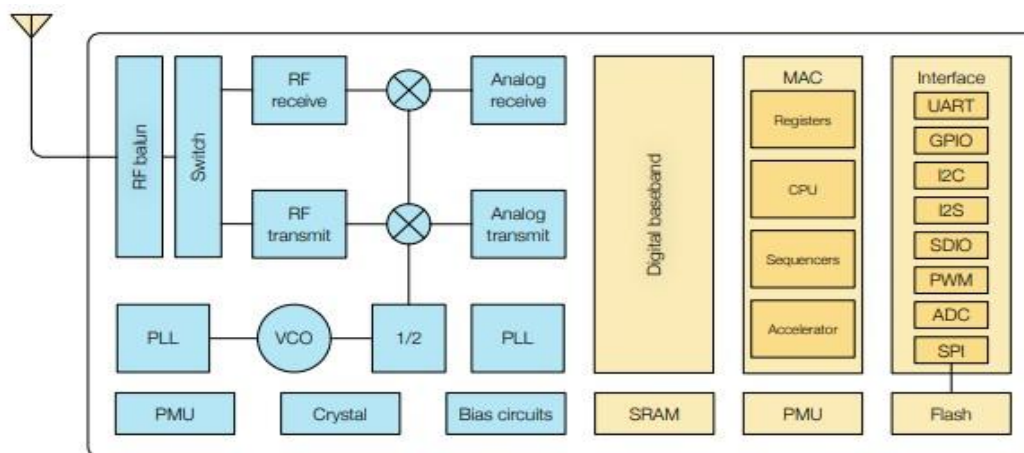


Fig 3.2.2-Architecture of ESP8266

PIN DESCRIPTION 8266

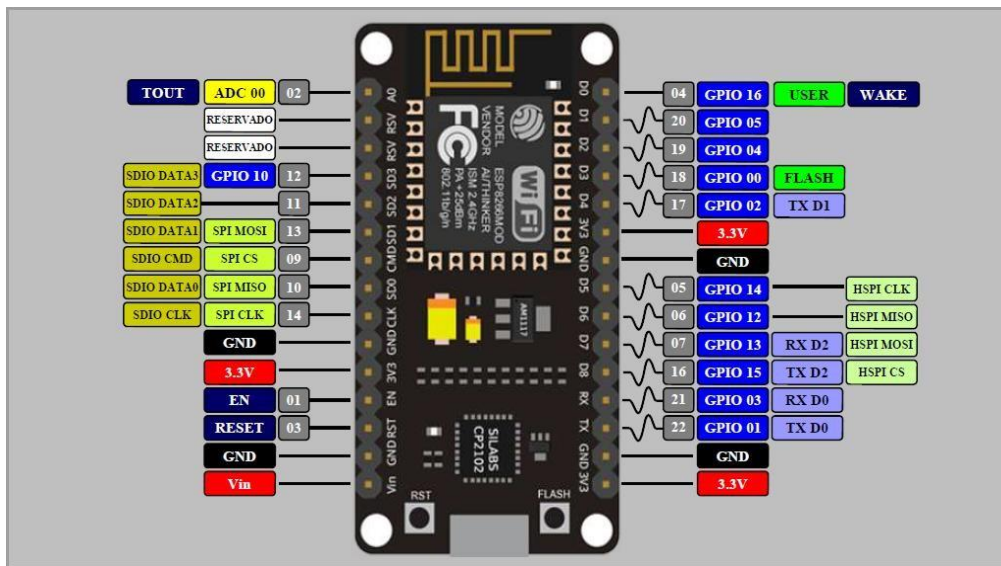


Fig 3.2.3-Pin Diagram of ESP8266 Board

ESP8266 FEATURES.

- 802.11 b / g / n
- Wi-Fi Direct (P2P), soft-AP
- Built-in TCP / IP protocol stack
- Built-in TR switch, balun, LNA, power amplifier and matching network
- Built-in PLL, voltage regulator and power management components
- 802.11b mode + 19.5dBm output power
- Built-in temperature sensor
- Support antenna diversity
- off leakage current is less than 10uA
- Built-in low-power 32-bit CPU: can double as an application processor
- SDIO 2.0, SPI, UART
- STBC, 1×1 MIMO, 2×1 MIMO
- A-MPDU, A-MSDU aggregation and the 0.4 Within wake
- 2ms, connect and transfer data packets
- standby power consumption of less than 1.0mW (DTIM3)

3.3 SERVO MOTOR (SG90 / MG995)

A **servo motor** is a highly precise and controlled motor commonly used in automation, robotics, and embedded systems to achieve accurate angular movement. In projects such as smart medicine dispensers, servo motors like the **SG90** and **MG995** play a crucial role by enabling controlled rotation of components such as dispensing wheels, lids, or mechanical arms. Unlike regular DC motors, servo motors do not rotate continuously; instead, they can rotate to a specific angle based on an input control signal. This makes them ideal for applications where precision, repeatability, and stable motion are essential.

The **SG90 servo motor**, often referred to as a **micro servo**, is small, lightweight, and suitable for low-load applications. It typically operates at **4.8V–6V** and provides a torque of around **1.8 kg/cm**, making it ideal for compact systems that need precise but light mechanical movement. Its plastic gear construction keeps it affordable and efficient for small mechanisms like rotating small medicine compartments. The SG90 offers an angular rotation of approximately **0° to 180°**, controlled through PWM (Pulse Width Modulation) signals, where the pulse duration determines the motor's position.



Fig.3.3.1 Servo motor

On the other hand, the **MG995 servo motor** is a **high-torque, metal-gear** servo designed for more demanding applications requiring strength and durability. Operating typically at **6V**, it can deliver torque of **10–13 kg/cm**, making it suitable for heavier loads or larger mechanical structures. Its internal metal gear system provides stability, high durability, and resistance to wear, making it reliable for continuous operation. Like the SG90, the MG995 also uses PWM signals to control its angular movement, offering fast response, high accuracy, and strong holding torque.

In a smart medicine dispenser, servo motors are used to **rotate the medicine dispensing wheel**, align the correct compartment with the outlet, lift a flap, or operate other mechanical parts with precise timing. The servo receives commands from a microcontroller such as **Arduino, ESP8266, or ESP32**, which sends the appropriate PWM signals based on programmed schedules or sensor feedback. Because servo motors provide controlled and repeatable movement, they ensure that medicines are dispensed accurately without jamming, misalignment, or mechanical errors.

Both SG90 and MG995 operate with minimal power consumption and offer smooth motion, making them ideal for embedded IoT devices. The choice between them depends on load requirements: **SG90 for lightweight, small movements**, and **MG995 for high-torque, heavy-duty operations**. Their ease of interfacing, low power needs, fast response, and high precision make these servo motors an indispensable part of automated systems, robotics projects, and smart healthcare devices.

3.4 BUZZER + LED INDICATORS



Fig.3.4.1. Buzzer

History

The history of an electromechanical buzzer and piezoelectric is discussed below.

Electromechanical

This buzzer was launched in the year 1831 by an American Scientist namely Joseph Henry but, this was used in doorbells until they were eliminated in 1930 in support of musical bells, which had a smooth tone.

Piezoelectric

These buzzers were invented by manufacturers of Japanese & fixed into a broad range of devices during the period of 1970s – 1980s. So, this development primarily came due to cooperative efforts through the manufacturing companies of Japanese. In the year 1951, they recognized the Application Research Committee of Barium Titanate that allows the corporations to be cooperative competitively & bring about numerous piezoelectric creations.

Specifications

The specifications of the buzzer include the following. • Color is black • The frequency range is 3,300Hz • Operating Temperature ranges from – 20° C to +60°C • Operating voltage ranges from 3V to 24V DC • The sound pressure level is 85dBA or 10cm • The supply current is below 15mA .

Types of Buzzer

A buzzer is available in different types which include the following.

- Piezoelectric
- Electromagnetic
- Mechanical
- Electromechanical
- Magnetic

Piezoelectric

As the name suggests, the piezoelectric type uses the piezoelectric ceramic's piezoelectric effect & pulse current to make the metal plate vibrate & generate sound. This kind of buzzer is made with a resonance box, multi resonator, piezoelectric plate, housing, impedance matcher, etc. Some of the buzzers are also designed with LEDs. The multi resonator of this mainly includes ICs and transistors. Once the supply is given to this resonator, it will oscillate and generates an audio signal with 1.5 to 2.kHz. The impedance matcher will force the piezoelectric plate to produce sound .

Electromagnetic

This type of buzzer is made with a magnet, solenoid coil, oscillator, housing, vibration diaphragm, and magnet. Once the power supply is given, the oscillator which produces the audio signal current will supply throughout the solenoid coil to generate a magnetic field. Sometimes, the vibration diaphragm will vibrate & generates sound under the magnet & solenoid coil interaction. The frequency range of this ranges from 2 kHz to 4kHz .

Working Principle

The working principle of a buzzer depends on the theory that, once the voltage is given across a piezoelectric material, then a pressure difference is produced. A piezo type includes piezo crystals among two conductors. Once a potential disparity is given across these crystals, then they thrust one conductor & drag the additional conductor through their internal property. So this continuous action will produce a sharp sound signal.

Mounting Configurations

The mounting configurations of buzzers include the following.

- Panel Mount
- Wire Leads
- Screw Terminals
- Through Hole
- Spring Contact
- Surface Mount

How to use a Buzzer?

A buzzer is an efficient component to include the features of sound in our system or project. It is an extremely small & solid two-pin device thus it can be simply utilized on breadboard or PCB. So in most applications, this component is widely used. There are two kinds of buzzers commonly available like simple and readymade. Once a simple type is power-driven then it will generate a beep sound continuously. A readymade type looks heavier & generates a Beep. Beep. Beep. This sound is because of the internal oscillating circuit within it.

LED INDICATOR



Fig.3.4.2. LED Indicator

An LED (Light Emitting Diode) indicator is a simple yet essential electronic component used to visually display the status or operation of a system. It works on the principle of electroluminescence, where the diode emits light when an electric current passes through it in the forward-biased direction. LED indicators are widely used in embedded systems, IoT devices, consumer electronics, control panels, and medical equipment because they provide instant visual feedback with very low power consumption.

In many electronic systems, LED indicators serve as status flags, helping users quickly understand system conditions such as power ON/OFF, charging status, fault detection, connectivity status, or operational alerts. LEDs come in different colors—such as red, green, blue, and yellow—each representing a specific function. For example, a green LED typically indicates normal operation, a red LED may indicate an error or low battery, and a blue LED can represent Bluetooth connectivity. Their fast response time, high brightness, and long lifespan make them ideal for continuous monitoring.

LED indicators are driven using a microcontroller or a simple circuit where a resistor is used in series to limit the current and protect the LED from damage. In IoT-based systems like a smart medicine dispenser, the LED indicator plays a vital role by notifying users about medication schedules, compartment activation, missed doses, or system connectivity to Wi-Fi. This visual cue makes the system more user-friendly and reliable, especially for elderly users who may depend on simple, clear signals for daily medication reminders.

3.5 POWER SUPPLY (5V/9V LISTED)

The **power supply unit**, typically rated at **5V or 9V**, is one of the most essential components in any embedded or IoT-based system, as it provides stable and regulated electrical power to all the electronic modules. In a smart medicine dispenser or similar automation projects, the power supply ensures that components such as the microcontroller, sensors, servo motors, displays, and communication modules operate reliably without fluctuations. A regulated power supply converts the high-voltage AC from the mains into a safe, low-voltage DC output that matches the requirements of the circuit. Most IoT devices commonly use **5V** for modules like Arduino UNO, ESP8266/ESP32, servo motors, sensors, and indicators, while **9V adapters** or batteries are often used when higher voltage input is needed before further regulation using onboard linear regulators.

The **5V power supply** is widely used because many digital electronics and microcontroller boards operate efficiently at this voltage. It provides clean, consistent power that prevents overheating, resets, or unexpected behavior in sensitive components. For example, modules such as RTC (DS3231/DS1307), LEDs, IR sensors, buzzers, and small servo motors like the SG90 require stable 5V input for accurate operation. On the other hand, **9V power supplies**—either through 9V adapters or batteries—are useful when the system requires slightly higher input voltage that can be stepped down internally using voltage regulators such as **7805**, buck converters, or onboard voltage regulation circuits present on development boards. This ensures flexibility in supplying power to different parts of the system based on their voltage requirements.

In an automated medicine dispenser, the power supply plays a critical role by ensuring uninterrupted operation of all time-critical processes. Components like the RTC need constant power to maintain time, ESP8266 modules require stable voltage for Wi-Fi connectivity, and servo motors need adequate current to perform mechanical movements without stalling. A good power supply must therefore provide not just the correct voltage, but also sufficient **current capacity** to handle peak loads. For instance, servo motors draw more current when rotating under load, and if the power supply is weak or unstable, the system may malfunction, skip doses, or lose connectivity.

To achieve reliability, regulated power supply modules often include features such as **voltage regulation, filtering capacitors, rectifier diodes, and protection circuits**. These components help eliminate electrical noise, reduce voltage ripple, and safeguard the system against sudden spikes or overloads. Many designs also incorporate a backup power option such as a rechargeable battery or power bank to ensure that critical operations, like RTC timekeeping or scheduled dispensing, are not interrupted during power failures.

Overall, the **5V/9V power supply** is the backbone of the entire system, providing clean, stable, and continuous energy to ensure smooth functioning of all connected electronic modules. Its reliability directly impacts the accuracy, safety, and performance of the device, making it a crucial component in any embedded or IoT-based project, especially those involving healthcare and automation.

3.6 MEDICINE DISPENSING WHEEL (3D PRINTED / ACRYLIC)

The **medicine dispensing wheel**, fabricated using **3D-printed material or acrylic sheets**, is a precisely designed circular structure that plays a central role in automated medication management by organizing tablets into separate, clearly defined compartments that correspond to specific time intervals or dosage schedules. This wheel is engineered with evenly spaced slots that ensure each medicine dose is stored securely, protected from contamination, and dispensed only when required.



Fig.3.6.1. Medicine dispensing wheel

The use of **3D printing** allows for customization in terms of compartment size, shape, and capacity, making it adaptable for different medication types, while **acrylic construction** provides high durability, transparency, and smooth movement during rotation. The wheel is mounted on a central axis and integrated with a **servo motor or stepper motor**, which rotates it accurately under microcontroller commands to align the correct compartment with the dispensing outlet at the right time. This controlled rotation mechanism ensures that medicines are delivered with precision, reducing the chances of human error such as missed doses or incorrect timing. Additionally, the wheel's design allows users or caregivers to pre-load medications for several days, making it highly convenient for elderly patients, individuals with memory issues, or those requiring strict adherence to daily medication routines. The smooth, low-friction surface of acrylic or the fine resolution of 3D printing ensures that tablets do not get crushed, stuck, or contaminated during storage and dispensing. The wheel's modular nature also makes it easy to clean, refill, and maintain, thereby enhancing hygiene and long-term usability. When integrated into a smart medicine dispenser, this wheel works in coordination with sensors, microcontrollers, alarms, and IoT-enabled reminders to provide a reliable, automated, and user-friendly solution for improving medication adherence and overall healthcare management.

3.7 RTC MODULE (DS3231 or DS1307)

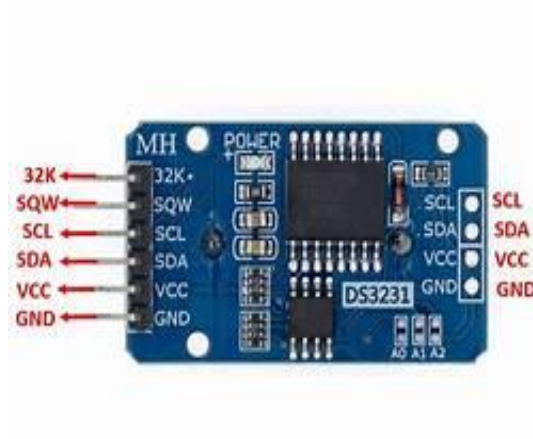


Fig.3.7.1 RTC Module

The **RTC (Real Time Clock) module**, such as the **DS3231** or **DS1307**, is an essential component used in embedded systems to maintain accurate timekeeping even when the main device is powered off. These modules are designed to track **seconds, minutes, hours, day**,

date, month, and year continuously, providing a reliable clock reference for applications that require precise scheduling. The DS3231 and DS1307 operate using an internal quartz crystal and a backup battery (usually a coin-cell battery), which ensures that the clock continues to run even during power interruptions. This feature makes RTC modules highly suitable for automation projects, data-logging systems, security devices, and smart medical dispensers where accurate time is critical. The **DS3231** is known for its high accuracy because it contains a **temperature-compensated crystal oscillator (TCXO)**, which adjusts the timekeeping based on temperature variations. This enables the DS3231 to maintain time with an accuracy of ± 2 minutes per year, making it one of the most precise RTC modules available. On the other hand, the **DS1307** is a widely used RTC due to its simplicity and low cost, though it is slightly less accurate because it depends on an external crystal oscillator that can drift with temperature changes. Both modules communicate with microcontrollers such as **Arduino, ESP8266, ESP32, Raspberry Pi**, and similar platforms through the **I²C (Inter-Integrated Circuit)** communication protocol, which uses only two lines (SDA for data and SCL for clock), making them easy to interface and integrate.

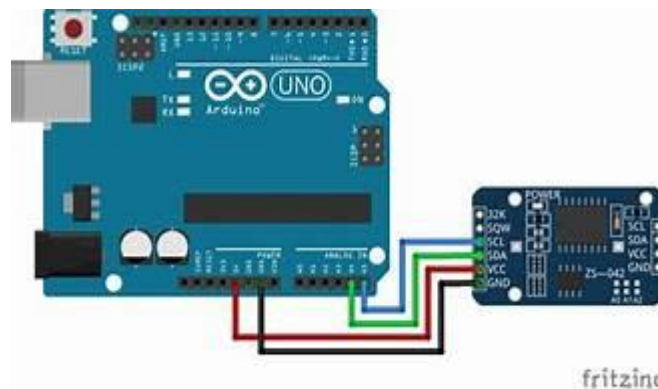


Fig.3.7.2. Interfacing MPU6050 with Arduino uno

In systems like a **smart medicine dispenser**, the RTC module plays a critical role by providing precise timing for scheduled alerts and automated medicine dispensing. The RTC allows the device to trigger alarms, rotate the dispensing wheel, activate LEDs or buzzers, and send notifications at accurate time intervals without relying on the microcontroller's internal clock, which is often unstable.

The backup battery ensures that the medication schedule is not lost during power failures, maintaining reliability for elderly users or patients who depend on timely medication.

Additionally, the RTC helps synchronize recorded events, such as logging the time of each medicine dispensed or missed doses, which is crucial for monitoring medication adherence.

Overall, the **RTC module** provides stable, long-term timekeeping, reducing dependency on software-based timing and ensuring accuracy in time-critical applications. Its low power consumption, easy interfacing, backup battery support, and accurate clock functionality make it a vital component in IoT devices, automation systems, wearables, and medical electronics. Whether using the highly accurate DS3231 or the affordable DS1307, the RTC ensures uninterrupted and dependable timekeeping essential for any system requiring precise scheduling and real-time operation.

3.8 16x2 LCD Display (with I2C module)

The LCD panel used in this block interfaced with micro-controller through output port. This is a 16 character $\times$ 2Line LCD module, capable of display numbers, characters, and graphics. The display contains two internal byte-wide registers, one for commands (RS=0) and the second for character to be displayed (RS=1). It also contains a user programmed Ram area (the character RAM) character that can be formed using dot matrix that can be programmed to generate any desired. Two distinguished between these areas, the hex command byte will be signify that the display RAM address 00h is chosen.

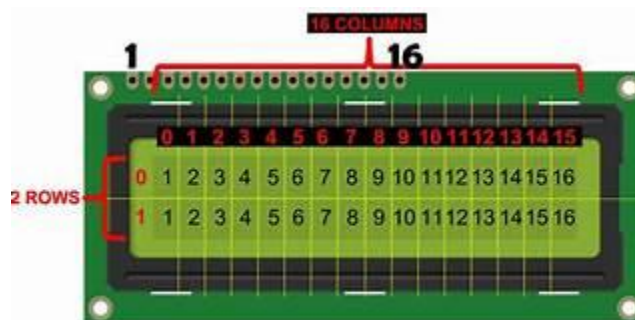


Fig.3.8.1 LCD Display

LCD can add a lot to our application in terms of providing a useful interface for the user, debugging an application or just giving it a “professional” look. The most common type of LCD controller is the Hitachi 44780 which provides a relatively simple interface between a processor and an LCD. Using this inter is often not attempted by inexperienced designers and

programmers because it is difficult to find good documentation on the interface, initializing the interface can be problem and the displays themselves are expensive. Connection to a PC parallel port is mostly simple. These displays can handle eight bit input directly. They also need two extra lines to control which kind of data has just arrived and when the data is meant to be stable. Those signals are also called RS (Register Select, instruction or data register) and EN (enable). So it has to control ten data lines (8 bits + RS + EN) and one common ground (GND) line, which make eleven lines to the parallel port. Data read back is not supported by the driver and so it does not require extra line for this.

LCD PIN DIAGRAM

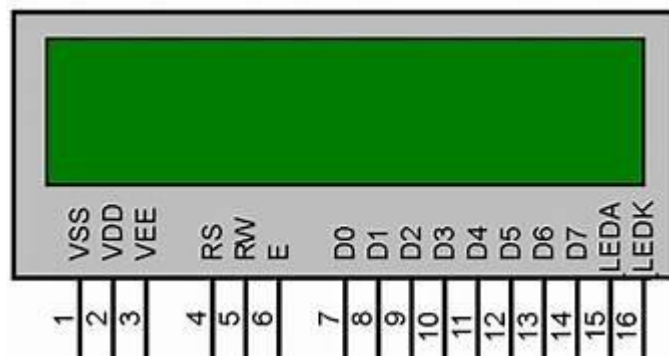


Fig 3.8.2-Pin diagram Of LCD 16X2

Features of 16×2 LCD module

- Operating Voltage is 4.7V to 5.3V
- Current consumption is 1mA without backlight
- Alphanumeric LCD display module, meaning can display alphabets and numbers
- Consists of two rows and each row can print 16 characters.
- Each character is build by a 5×8 pixel box
- Can work on both 8-bit and 4-bit mode
- It can also display any custom generated characters
- Available in Green and Blue Backlight

3.9 PUSH BUTTON

A push button is a simple, manually operated switch used to control electronic circuits by making or breaking an electrical connection when pressed. It is one of the most fundamental input devices in embedded systems, automation projects, and IoT-based devices because it provides a reliable and direct way for users to interact with the system. A push button works on a mechanical principle: when the button is pressed, its internal metal contacts touch each other and complete the circuit (usually referred to as the "pressed" or "ON" state), and when it is released, the spring mechanism separates the contacts, returning the circuit to the "open" or "OFF" state. This momentary operation allows the button to generate short, controlled electrical signals that can be detected by microcontrollers like Arduino, ESP8266, or ESP32. In a smart medicine dispenser, the push button serves as an essential user interface element that allows users or caregivers to perform specific actions such as acknowledging reminders, confirming that a dose has been taken, navigating menu options on the display, resetting the system, or manually triggering the dispensing mechanism. Because push buttons provide tactile feedback, users can clearly feel when the button is pressed, which is especially helpful for elderly individuals who may have difficulty with touchscreens or small controls. The tactile nature ensures reliable operation and avoids accidental activation.

Push buttons are typically connected to the microcontroller using either pull-up or pull-down resistors, which ensure that the input pin reads a stable logic HIGH or LOW value when the button is not pressed. This prevents noise or floating signals from causing false triggering. When the button is pressed, the electrical state changes, and the microcontroller detects this change in logic level as an event. Debouncing techniques—either through hardware capacitors or software algorithms—are often applied to ensure the button press is accurately recorded, eliminating rapid fluctuations caused by mechanical contact bounce.

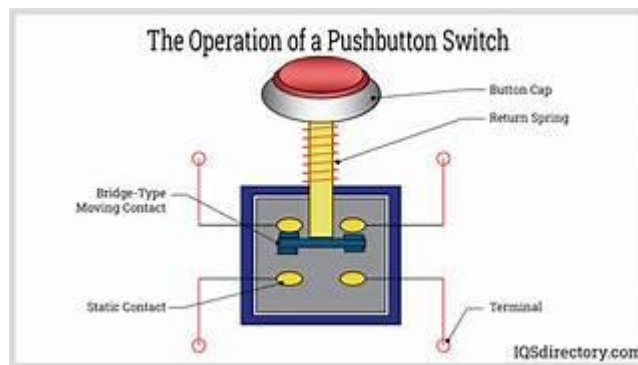


Fig.3.9.1 Push button

Push buttons can be made from different materials such as plastic, metal, or rubber, and come in various sizes and shapes depending on the application. In enclosure-based systems like medicine dispensers, the buttons are mounted on the outer surface to ensure easy access while keeping internal circuits protected. The button cap is designed for comfortable pressing, while the underlying switch is rated for thousands of press cycles, ensuring long-term durability.

CHAPTER 4

SOFTWARE DESCRIPTION

4.1. BLYNK / FIREBASE / MQTT IOT MOBILE APP

The IoT Mobile App, developed using platforms like Blynk, Firebase, or MQTT, serves as the central interface for monitoring, controlling, and managing smart devices remotely. These platforms enable seamless communication between embedded hardware (such as Arduino, ESP8266, ESP32, or Raspberry Pi) and the user's smartphone through cloud connectivity. In smart healthcare systems like automated medicine dispensers, the app plays a vital role by providing real-time updates, sending reminders, delivering notifications, and allowing remote configuration of medication schedules. offers a simple drag-and-drop mobile dashboard where buttons, gauges, notifications, graphs, and timers can be placed to control the device without writing complex UI code. The hardware communicates with the Blynk Cloud using authentication tokens, allowing the user to check device status, trigger manual medicine dispensing, view logs, and receive alerts instantly. Its user-friendly interface makes it ideal for rapid IoT development and real-time interaction.

Firebase, developed by Google, provides a robust backend with features such as Real-Time Database, Firestore, Authentication, and Cloud Messaging. In an IoT healthcare device, Firebase enables secure storage of dispensing records, missed dose alerts, user profiles, and device information.

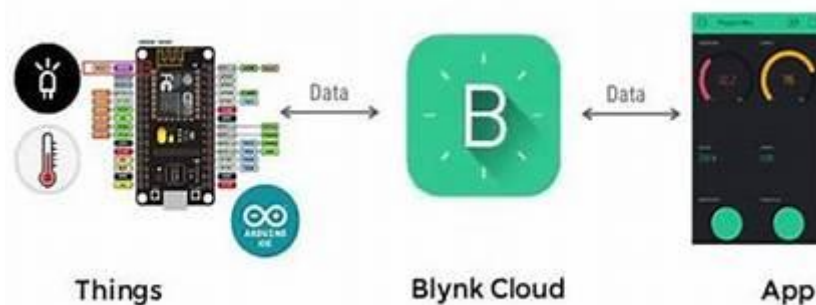


Fig.4.1.1.Blynk / firebase / mqtt iot mobile app

MQTT (Message Queuing Telemetry Transport) is a lightweight publish–subscribe protocol designed for fast, efficient IoT communication. It ensures that real-time updates—such as “medicine dispensed,” “compartment empty,” or “missed dose”—are delivered instantly from the device to the app. MQTT works efficiently even in low-bandwidth or unstable network conditions, making it dependable for continuous healthcare monitoring. The MQTT broker manages messages between the device and the app, ensuring low latency and reliable delivery.

Together, Blynk, Firebase, and MQTT provide a complete IoT mobile app ecosystem that supports device control, cloud storage, real-time analytics, and dependable message communication. These platforms enhance the convenience, accuracy, and reliability of smart systems, especially in healthcare, by ensuring the user is always informed and connected to the device. They enable automation, improve user safety, and make the overall system more intelligent and user-friendly.

4.2. EMBEDDED C / C++

Embedded C and Embedded C++ are specialized programming languages used to develop software for embedded systems—small, dedicated computing units integrated into devices such as smart medicine dispensers, IoT devices, home appliances, automotive controllers, and industrial machines. Unlike general-purpose computers, embedded systems operate with limited memory, processing power, and strict real-time requirements; therefore, the programming languages used must be efficient, fast, and hardware-oriented. Embedded C extends the standard C language with additional features that allow direct hardware access, memory manipulation, and timing control, making it ideal for microcontrollers such as the ATmega328, PIC, ARM Cortex, ESP8266, and STM32. It supports operations like configuring registers, controlling digital I/O pins, reading sensors, and generating precise timing delays, which are essential for real-time embedded applications. Embedded C++ is an adaptation of C++ designed for embedded platforms. While standard C++ offers powerful object-oriented features, it can be heavy for resource-constrained microcontrollers. Embedded C++ restricts unnecessary features like exceptions, templates, and multiple inheritance but retains essential object-oriented capabilities such as classes and objects, allowing better modularity and maintainability. This makes it particularly useful for large embedded applications where code organization, scalability, and reusability matter.

Both Embedded C and C++ allow developers to work close to the hardware using low-level programming techniques. This enables direct manipulation of microcontroller registers to control peripherals such as timers, UART, I2C, ADC, and PWM modules. Developers can interact with sensors, actuators, motors, and communication modules by writing precise instructions that execute within microseconds. Since embedded systems often require deterministic timing, Embedded C/C++ provides mechanisms to work with interrupts, handle timing constraints, and ensure the system responds instantly to external events. Another advantage is portability. Programs written in Embedded C/C++ can be migrated across different microcontrollers with minimal changes, provided the hardware abstraction layers are properly designed. These languages also integrate well with real-time operating systems (RTOS), enabling multitasking and improved system reliability. Because embedded devices often operate continuously and perform safety-critical functions, Embedded C/C++ code must be optimized, reliable, and memory-efficient. Programmers must carefully manage stack usage, RAM allocation, and instruction cycles. This leads to the creation of stable firmware that can run for years without failure.

In summary, Embedded C and Embedded C++ form the backbone of modern embedded system development. They provide the performance, control, and efficiency needed to interact directly with hardware while supporting advanced features for scalable system design. Their widespread use in IoT devices, automation systems, healthcare equipment, and consumer electronics highlights their importance in developing smart, reliable, and real-time embedded solutions.

4.3. CLOUD DATABASE

A **cloud database** is a database service that runs on cloud computing platforms such as AWS, Google Cloud, Microsoft Azure, Firebase, and other online infrastructures. Unlike traditional databases that are installed and managed on a local computer or on-premise server, cloud databases are hosted on remote data centers and accessed over the internet. This provides scalability, reliability, and flexibility, making cloud databases ideal for modern IoT systems, mobile applications, and real-time data-driven projects like smart medicine dispensers. Cloud databases allow users to store, retrieve, and manage data without worrying about hardware maintenance, storage limitations, or system failures. Service providers handle all backend

operations such as automatic backups, software updates, security patches, and disaster recovery. Because of this, developers and organizations can focus entirely on application development instead of managing physical servers. Additionally, cloud databases are accessible from any location, enabling seamless integration between devices, web interfaces, and mobile apps.

There are two major types of cloud databases—**SQL (structured)** and **NoSQL (unstructured)**. SQL cloud databases like Amazon RDS or Google Cloud SQL follow a table-based structure and are best suited for applications requiring structured data and strong relationships. NoSQL databases like Firebase Realtime Database, MongoDB Atlas, or Amazon DynamoDB store data in JSON-like formats and offer high performance for real-time updates, making them ideal for IoT and mobile applications. For example, in a smart medicine box, sensor readings, user reminders, medicine schedules, and device status are uploaded to a cloud database in real time so that the mobile app can instantly reflect changes. A key advantage of cloud databases is **scalability**. As the number of users or devices increases, the database automatically expands to accommodate more data and traffic. This is especially useful in IoT systems where thousands of devices may simultaneously send data. Another major advantage is **high availability**—cloud databases distribute data across multiple servers and geographic regions, ensuring the system remains accessible even during hardware failures or network issues.

In summary, cloud databases offer a powerful, cost-effective, and flexible solution for storing and managing data for modern applications. Their ability to provide real-time access, automatic scaling, reliability, and easy integration with IoT and mobile apps makes them an essential backbone for smart systems and connected devices. By eliminating the need for physical servers and offering strong security features, cloud databases help developers build robust and user-friendly applications with minimal infrastructure effort.

4.4. WI-FI CONNECTION

A Wi-Fi connection is a wireless networking technology that allows electronic devices such as smartphones, laptops, IoT modules, microcontrollers, and smart home appliances to communicate with each other and access the internet without using physical cables. Wi-Fi is based on IEEE 802.11 standards and operates using radio frequency signals, typically in the 2.4 GHz and 5 GHz bands. This wireless technology has become the foundation of modern connectivity because it provides high-speed data transfer, mobility, and convenience for both personal and industrial applications. In IoT systems, a Wi-Fi connection plays a crucial role by enabling microcontroller-based devices (such as ESP8266, ESP32, or Raspberry Pi) to send and receive data over the internet.

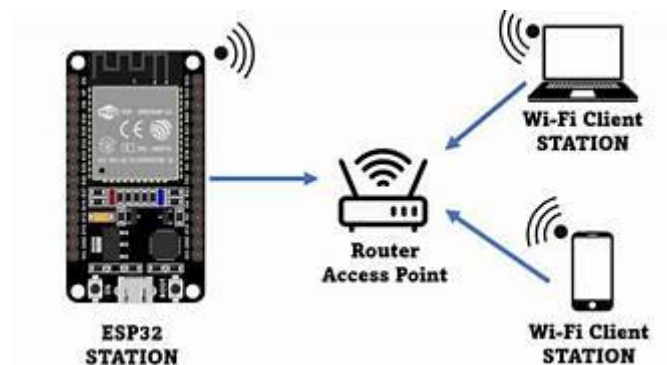


Fig.4.4.2. wifi connection

Through Wi-Fi, these devices can upload sensor readings, receive commands from cloud servers, and interact with mobile applications. For example, in a smart medicine dispenser, Wi-Fi allows the device to communicate with cloud databases like Firebase or MQTT broker, ensuring real-time updates of medicine schedules, notifications, and user interactions. The Wi-Fi module connects to the local router using SSID (network name) and password authentication, forming a link between the embedded hardware and cloud. A Wi-Fi network typically consists of a router or access point that acts as the central hub. The router broadcasts signals within a specific range, usually between 20 to 50 meters indoors. Devices equipped with Wi-Fi receivers can detect these signals and join the network by providing the correct credentials.

Once connected, data is transferred using radio waves that are encoded, transmitted, received, and decoded by network interfaces. Wi-Fi supports different security protocols such as WPA2, WPA3, and AES encryption, ensuring safe and secure communication over the wireless network. Wi-Fi also supports IP communication, meaning each connected device receives an IP address through DHCP. This enables cloud platforms to uniquely identify the device, exchange commands, and ensure reliable communication. For embedded systems, libraries and firmware handle Wi-Fi tasks such as connecting to the router, maintaining stable connectivity, and handling reconnections if the network signal drops.

In summary, a Wi-Fi connection is an essential wireless communication technology that enables seamless data transfer between devices and the internet. Its speed, reliability, security, and easy integration make it an ideal choice for modern IoT projects and embedded applications. Whether used in smart appliances, healthcare devices, or industrial automation, Wi-Fi serves as the backbone for real-time monitoring, remote control, and cloud interaction.

4.5 SOURCE CODE

```
#include <ESP8266WiFi.h>

#include <ESP8266WebServer.h>

#include <Wire.h>

#include <RTCLib.h>

#include <Servo.h>

#include <LiquidCrystal_I2C.h>

#include <NTPClient.h>

#include <WiFiUdp.h>


// WiFi Credentials

const char* ssid = "YourWiFiName";

const char* password = "YourWiFiPassword";


// RTC, LCD, and Servo objects

RTC_DS3231 rtc;

LiquidCrystal_I2C lcd(0x27, 16, 2);

Servo servoMotor;

ESP8266WebServer server(80);


// NTP Client setup

WiFiUDP ntpUDP;

NTPClient timeClient(ntpUDP, "pool.ntp.org", 19800, 60000); // 19800 = IST offset in seconds
```

```
// I/O Pins

#define BUZZER D5

#define BUTTON D6

#define SERVO_PIN D7


// Medicine times (24-hour format)

int medHour[3] = {8, 13, 20};

int medMinute[3] = {0, 0, 0};


bool medDispensed[3] = {false, false, false};


// Web page form for setting medicine times

String htmlPage() {

    String page = "<html><body><h2>Smart Medicine Dispenser</h2>"

        "<form action='/setTimes' method='GET'>"

        "Medicine 1 Time (HH:MM): <input name='t1'><br>"

        "Medicine 2 Time (HH:MM): <input name='t2'><br>"

        "Medicine 3 Time (HH:MM): <input name='t3'><br><br>"

        "<input type='submit' value='Set Times'></form></body></html>";

    return page;

}


// Convert time string (HH:MM) to hour and minute
```



```
void parseTime(String timeStr, int &hour, int &minute) {  
  
    int colonIndex = timeStr.indexOf(':');  
  
    if (colonIndex > 0) {  
  
        hour = timeStr.substring(0, colonIndex).toInt();  
  
        minute = timeStr.substring(colonIndex + 1).toInt();  
  
    }  
  
}  
  
  
// Handle web form submission  
  
void handleSetTimes() {  
  
    if (server.hasArg("t1")) parseTime(server.arg("t1"), medHour[0], medMinute[0]);  
  
    if (server.hasArg("t2")) parseTime(server.arg("t2"), medHour[1], medMinute[1]);  
  
    if (server.hasArg("t3")) parseTime(server.arg("t3"), medHour[2], medMinute[2]);  
  
    server.send(200, "text/html", "<h3>Times updated successfully!</h3><a href='/'>Go  
Back</a>");  
  
}  
  
  
void setup() {  
  
    Serial.begin(115200);  
  
    WiFi.begin(ssid, password);  
  
    lcd.init();  
  
    lcd.backlight();  
  
    servoMotor.attach(SERVO_PIN);  
  
    pinMode(BUZZER, OUTPUT);
```

```
pinMode(BUTTON, INPUT_PULLUP);

digitalWrite(BUZZER, LOW);


lcd.setCursor(0, 0);

lcd.print("Connecting WiFi...");

while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

}

lcd.clear();

lcd.print("WiFi Connected!");

delay(1000);


timeClient.begin();

rtc.begin();


server.on("/", []() { server.send(200, "text/html", htmlPage()); });

server.on("/setTimes", handleSetTimes);

server.begin();


lcd.clear();

lcd.print("System Ready");

delay(1000);

}
```

```
void loop() {

    server.handleClient();

    timeClient.update();


    DateTime now = rtc.now();

    lcd.setCursor(0, 0);

    lcd.print("Time: ");

    lcd.print(now.hour());

    lcd.print(":");

    lcd.print(now.minute());

    lcd.print(" ");

    for (int i = 0; i < 3; i++) {

        if (now.hour() == medHour[i] && now.minute() == medMinute[i] && !medDispensed[i]) {

            lcd.setCursor(0, 1);

            lcd.print("Dispensing Slot ");

            lcd.print(i + 1);

            servoMotor.write(90); // Rotate to dispense

            digitalWrite(BUZZER, HIGH);


            // Wait for user confirmation

            while (digitalRead(BUTTON) == HIGH) {

                delay(100);
```

```
}  
  
digitalWrite(BUZZER, LOW);  
  
servoMotor.write(0);  
  
medDispensed[i] = true;  
  
lcd.clear();  
  
lcd.print("Slot ");  
  
lcd.print(i + 1);  
  
lcd.print(" done!");  
  
delay(2000);  
  
}  
  
// Reset daily  
  
if (now.hour() == 0 && now.minute() == 0) {  
    medDispensed[i] = false;  
}  
  
}  
  
}
```

CHAPTER 5

METHODOLOGY

The methodology outlines the step-by-step approach followed to design, develop, implement, and test the Smart Medicine Dispenser using ESP8266 IoT technology. The project adopts a structured flow starting from problem analysis to system deployment. The system begins with the Arduino Uno initializing all connected components, including the LCD display, Real-Time Clock (RTC) module, buzzer, and LEDs. Upon startup, the Arduino continuously reads the current time from the RTC and displays it on the LCD. A push button serves as user input; when pressed, it triggers specific actions like starting a timer, setting an alarm, or resetting the system. The Arduino continuously monitors the time, comparing it against any pre-set values. When the programmed condition (such as a specific time or duration) is met, the Arduino activates output devices: the buzzer sounds an audible alert, LEDs light up or blink as visual indicators, and the LCD shows a corresponding message. The system remains active until the user intervenes, typically by pressing the button again to stop the buzzer, turn off LEDs, and reset the system for the next cycle, ensuring efficient time-based control and notifications.

5.1 BLOCK DIAGRAM

The block diagram illustrates the complete operational workflow and communication architecture of the Smart Medicine Dispenser system. This design integrates software, cloud computing, IoT hardware, and mechanical dispensing components to ensure accurate and timely medication delivery. The system begins with the Mobile Application, which serves as the interface through which users or caregivers can interact with the device. Using the app, the user can set up medicine schedules, dosage timings, medicine names, and frequency of intake. The mobile app also provides reminders, notifications, and real-time feedback about whether the medicine has been taken or skipped. All data entered through the app is transmitted securely to the Cloud Server, ensuring that information is stored centrally and can be accessed or modified from anywhere.

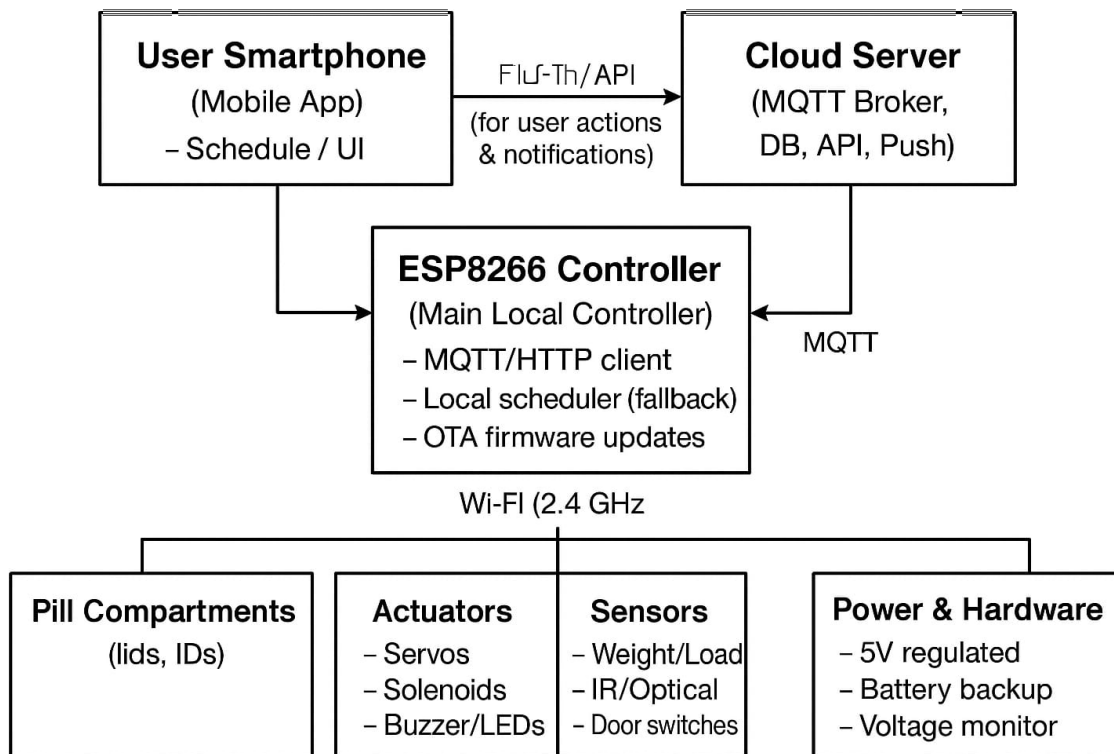


Figure.5.1.1.Block diagram

The block diagram illustrates the complete operational workflow and communication architecture of the Smart Medicine Dispenser system. This design integrates software, cloud computing, IoT hardware, and mechanical dispensing components to ensure accurate and timely medication delivery. The system begins with the Mobile Application, which serves as the interface through which users or caregivers can interact with the device. Using the app, the user can set up medicine schedules, dosage timings, medicine names, and frequency of intake. The mobile app also provides reminders, notifications, and real-time feedback about whether the medicine has been taken or skipped. All data entered through the app is transmitted securely to the Cloud Server, ensuring that information is stored centrally and can be accessed or modified from anywhere.

The cloud server acts as the primary controller of system data, managing medication schedules, analyzing user interactions, and coordinating communication with the device. It ensures that every schedule is processed correctly and sends commands to the hardware in real time. The Cloud Server then communicates with the ESP8266, a Wi-Fi-enabled microcontroller that acts as the central processing unit of the physical dispensing device. The ESP8266 continuously stays connected to the server, listening for schedule updates and operating commands. When a medication time is reached, the cloud sends a command to the ESP8266 to initiate the dispensing process. The ESP8266 is responsible for controlling both the electronic sensing components and the mechanical actuation units of the system. It activates different sensors to monitor the presence, movement, and release of medicine. These may include IR sensors to detect whether the medicine has been removed, weight sensors to measure remaining pill quantities, and other environmental sensors that ensure safe operation.

Once the dispensing procedure is triggered, the ESP8266 also controls the servo motor, which performs mechanical actions such as opening the correct medicine compartment. The servo precisely rotates or lifts the compartment lid, allowing the user to easily access the pills. The connection Sensors & Servo ↔ Medicine Compartments represents this coordinated action where sensors monitor the compartment and the servo ensures proper opening and closing at scheduled times. The presence of sensors helps verify whether the user has taken the medicine, whether compartments are empty, and whether any mechanical obstruction exists during operation. This information is immediately sent back to the ESP8266

5.2 CIRCUIT DIAGRAM

This circuit is built around the ESP8266 NodeMCU, which acts as the central microcontroller to manage all inputs and outputs. The system includes a 16×2 I2C LCD display, connected through the SDA and SCL lines, allowing the controller to show time, messages, or system status using minimal wiring. On the same I2C communication bus, a DS3231 Real-Time Clock (RTC) module is added to keep very accurate time even when the system is powered off, thanks to its built-in battery. This makes the setup suitable for scheduling tasks such as alarms, reminders, or timed dispensing.

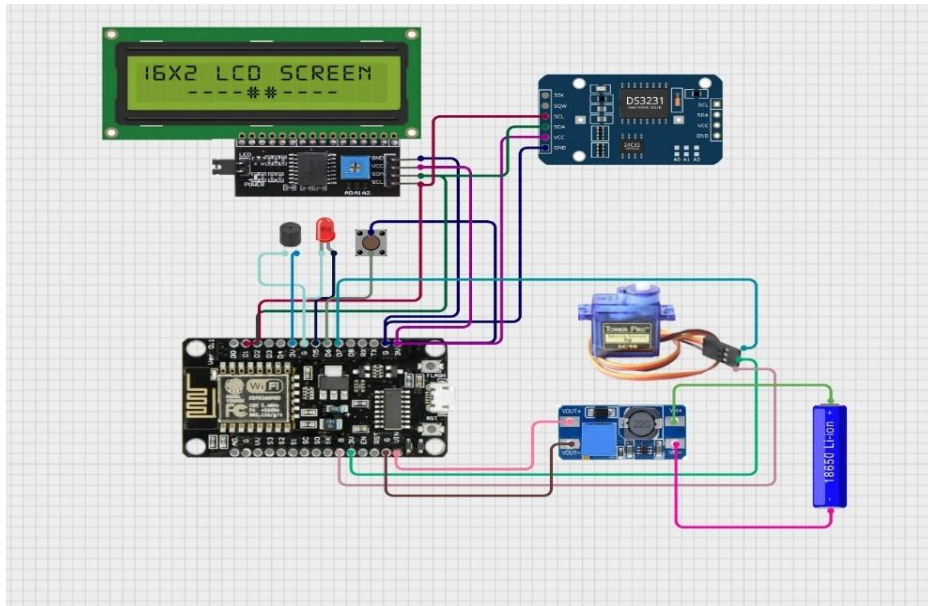


Fig.5.2.2.Circuit Diagram

A servo motor (Tower Pro SG90) is connected to one of the PWM-capable pins of the NodeMCU. Its power is supplied separately through a buck converter module, which steps down the voltage from the 18650 Li-ion battery to a stable 5 V output. This is important because the servo requires more current than what the NodeMCU can safely supply, and using a dedicated buck converter prevents brown-outs and resets. The ground of the buck converter is shared with the NodeMCU to maintain a common reference for signals.

Additional components include a push button, an LED, and a buzzer, each wired to appropriate GPIO pins with their respective ground or power connections. The push button can be used to set time, confirm actions, or trigger manual operations. The LED acts as a visual indicator for system states such as power-on, alarms, or errors, while the buzzer provides audible alerts, for example in medication reminders or timed notifications.

Overall, this circuit forms a battery-powered, time-based control system that can display information on the LCD, monitor accurate time using the DS3231, alert the user with LED and buzzer indicators, and perform mechanical actions using the servo. The NodeMCU's Wi-Fi capability can also be used to sync time online or send notifications, making the setup suitable for projects like smart medicine boxes, automatic feeders, alarms, and scheduling devices.

CHAPTER 6

RESULT AND DISCUSSION

6.1 FINAL OUTPUT MODEL

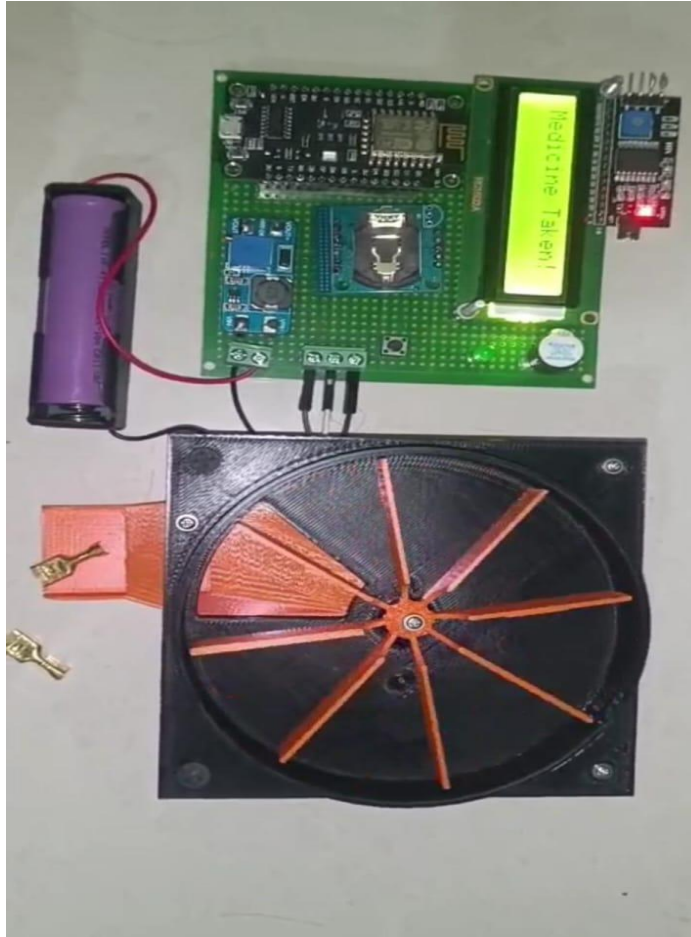


Fig.6.1.1. Final Output Model

The primary purpose of the **Intelligent Medicine Box** is to **help patients take their medicines on time and in the correct dose** by using **automation, IoT connectivity, and smart alerts**. It aims to **reduce missed doses, prevent over-medication, and improve medication adherence**, especially for **elderly people, chronically ill patients, and busy individuals**.

Through features like **real-time reminders, automatic compartment opening, mobile notifications, and monitoring by caregivers**, the system ensures safe and reliable medicine management.

6.2 RESULTS

The **Intelligent Medicine Box for Medication Management using IoT** was successfully built and tested. The system worked well and performed all the important functions as planned. The medicine box was able to remind users to take their medicines at the correct time through alarms, LEDs, and mobile notifications. During testing, the reminders were accurate and triggered exactly at the scheduled time. The servo motor and compartments operated smoothly, allowing the correct medicine section to open when the reminder was active. This helped reduce confusion and made medicine-taking easier for users, especially the elderly. The mobile app (Blynk/Firebase/MQTT) also worked properly and displayed real-time information, such as medicine timings, taken/missed doses, and device status.

The IoT connectivity remained stable, and users were able to monitor the medicine box from anywhere. Caregivers could also check if the user had taken their medicine on time, improving safety and ensuring better health management. The system ran continuously for several days without major issues, showing good reliability. Overall, the results show that the smart medicine box helped improve medication habits, reduced missed doses, and made it easier for users to manage their medicines independently. The project proved to be useful, reliable, and cost-effective for daily healthcare needs.



Fig.6.2.1. Result flow

6.3 ADVANTAGES

1.Remote Patient Monitoring: IoT devices enable continuous monitoring of patients' vital signs, allowing healthcare providers to remotely track and manage chronic conditions. This can reduce the need for frequent hospital visits and enhance the quality of care for patients with long-term health issues.

2.Improved Patient Engagement:IoT technologies encourage patients to actively participate in their healthcare. Wearable devices and mobile apps can provide patients with real-time health data, promote healthy behaviors, and enable them to take a more proactive role in managing their well-being.

3.Real-time Location Services: IoT can assist in tracking the location of patients, staff, and equipment within healthcare facilities. This can improve workflow efficiency, reduce wait times, and enhance overall patient experience.

4.Data-driven Decision Making: IoT generates vast amounts of data that, when properly analyzed, can provide valuable insights for healthcare providers and administrators.

6.4 DISADVANTAGES

1. Dependence on Internet Connectivity: The system requires a stable internet connection to send medication reminders, update cloud data, and sync information with the mobile application. If the Wi-Fi network is weak, interrupted, or unavailable, the device may fail to send timely alerts or synchronize user data.

2. High Initial Cost Compared to Traditional Boxes: An IoT-enabled medicine box includes sensors, microcontrollers, Wi-Fi modules, a power system, and cloud services, making it significantly more expensive than a regular pill organizer. For some users, especially in low-income households, this high initial cost can be a financial barrier, even though the system offers advanced features.

3. Requires Technical Knowledge for Setup and Use: Installing the app, connecting to Wi-Fi, configuring medicine timings, and understanding notifications may be difficult for elderly people or non-technical users. If users struggle with setup or operation, they may not fully benefit from the system and may require help from family members or caregivers for day-to-day use.

4. Risk of Data Privacy and Security Issues: Since the device stores sensitive health information on cloud platforms, there is always a risk of unauthorized access or data misuse. If proper encryption and security measures are not implemented, personal data such as medication schedules, health history, and user behaviour can become vulnerable to cyberattacks or privacy breaches.

5. Hardware or Sensor Failure Possibility: The device depends on electronic components such as servo motors, IR sensors, RTC module, and microcontroller. Over time, mechanical wear or environmental factors like dust, moisture, or low-quality components may cause failures. If sensors malfunction, the system may give incorrect notifications, fail to detect pill removal, or dispense medicine inaccurately.

6.5 APPLICATIONS

1.Medication Reminders: Intelligent medicine boxes can send notifications to patients, reminding them to take their medication at the right time.

2.Real-Time Data Analytics and Alerts: The data collected from these devices can be analyzed to track patterns in medication usage, helping healthcare professionals assess the effectiveness of treatments.

3.Security and Privacy: IoT-enabled boxes can have secure authentication features, such as biometric scanning, to ensure that the right patient is taking the right medication.

4.Personalized Medication Plans: Intelligent medicine boxes can be programmed with personalized medication schedules based on the patient's medical history and request.

6.6 APPROXIMATE COST**Table 6.6.1-Project Cost Approximation**

NO.	COMPONENT	QUANTITY	COST
1	ESP8266 NodeMCU	1	Rs.650
2	RTC Module	1	Rs.300
3	16x2 LCD Display	1	Rs.550
4	Buzzer	1	Rs.50
5	Push Button	1	Rs.150
6	Jumper wires,kit	—	Rs.800
7	Servo Motor	1	Rs.500
8	Medicine Slots /Container	1	Rs.1000
9	Power Supply	—	—
	TOTAL		4000

CONCLUSION

The “Smart Medicine Dispenser Using ESP8266 IoT” project successfully demonstrates an automated and reliable solution for managing patient medication schedules. It ensures timely dispensing, provides real-time alerts, reduces human error, and enhances healthcare efficiency. This system is highly suitable for elderly people and patients needing continuous monitoring, making it an effective low-cost healthcare automation product. Overall, the intelligent medicine box demonstrates the potential of IoT technology in transforming healthcare into a more connected, automated, and user-friendly ecosystem. It provides a step toward personalized healthcare solutions that promote responsible medication habits, improve patient outcomes, and support a healthier and more independent lifestyle.

FUTURE SCOPE

The **Intelligent Medicine Box for Medication Management Using IoT** has a wide scope for future improvements in both functionality and healthcare applications. In upcoming versions, advanced sensors such as weight sensors, pill counters, and biometric identification can be added to ensure accurate dispensing and prevent unauthorized access. Artificial Intelligence (AI) and machine learning algorithms can be integrated to analyze user behavior, predict missed doses, and offer personalized reminders based on individual medication patterns. The system can be expanded to support multiple users, making it suitable for hospitals, clinics, and elderly care centers. Future versions may incorporate GPS tracking for monitoring patients with memory disorders and a low-power or solar-powered design to improve energy efficiency. With advancements in cloud technology, secure data analytics can generate detailed health reports and dosage history. Overall, the future scope of this IoT-based medicine box is promising, with the potential to evolve into a complete smart healthcare assistant that improves medication adherence, patient safety, and remote health monitoring.

REFERENCES

1. “IoT Smart Medicine Box” Lee et al., —International Journal of Research in Engineering, Science and Management (IJRESM), 2018.
2. “Management System”.Patel et al., —International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET), 2019.
3. “Personalized Medication Systems”. Zhang et al., —Design of a Smart Medical Box for Automatic Pill Dispensing and Health Monitoring, 2021.
4. “Medication Management using Smart Pill Box and IoT”— International Journal of Engineering Research & Technology (IJERT), 2020.
5. “IoT Based Smart Medicine Box for Elderly People” — International Journal of Innovative Research in Computer and Communication Engineering (IJRCCE), 2021.
6. “Smart Medicine Box for Medication Management using IoT” — International Research Journal of Engineering and Technology (IRJET), 2020.
7. “A Review of Features and Characteristics of Smart Medication Adherence Products” S.Faisal et al., Journal of Medical Internet Research, 2021.
8. “Enhancing Medication Adherence with an IoT-Based Pill Dispenser and Smart Cup,” V.Peddisetti et al., International Conference on IoT Systems, 2024
9. “Technologies for Medication Adherence Monitoring and Support: Narrative Review,” M. Mason et al., JMIR mHealth and uHealth, 2022.
10. “IoT-Based Smart Medicine Box: Design and Implementation,” S. Patel et al., Inter